SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 Total Marks: 50 Duration: 60 Minutes Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I __A.Shiva Shankar Reddy(192425174)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____ A.Shiva Shankar Reddy_____

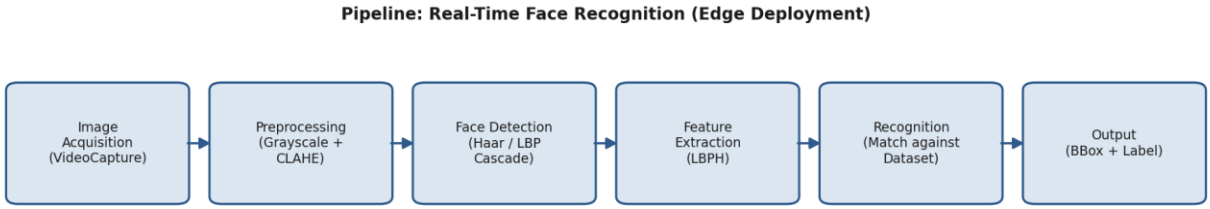
Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks (100)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Real-Time Face Recognition System (Edge Deployment)

A company wants to deploy a face recognition system on low-power edge devices (e.g., Raspberry Pi). Constraints: (A) Real-time processing (<30 FPS latency), (B) Varying lighting conditions, (C) Limited memory and CPU resources, (D) Detect and recognize faces from a predefined dataset.



Stage 1: Image Acquisition

Live video is captured using cv2.VideoCapture(0) with resolution deliberately reduced (e.g., 320×240) instead of full HD. A threaded frame-reader (cv2.VideoCapture wrapped in a Python thread / imutils.VideoStream) is used so that I/O wait does not block the CPU-bound detection stage. Frame skipping (processing every 2nd or 3rd frame) further reduces load on the edge device.

Stage 2: Preprocessing

Each frame is converted to grayscale using cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY), which cuts the data volume to one-third and speeds up all subsequent operations. To counter varying lighting, Contrast Limited Adaptive Histogram Equalization is applied using cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8)).apply(gray). CLAHE normalizes local contrast without over-amplifying noise, which is more robust than a simple cv2.equalizeHist() under uneven illumination such as backlighting or shadows.

Stage 3: Face Detection

A Haar Cascade classifier (cv2.CascadeClassifier('haarcascade_frontalface_default.xml')) or an LBP cascade is used with detectMultiScale(scaleFactor=1.1, minNeighbors=5). These classical detectors run efficiently on CPU-only edge hardware, unlike deep CNN detectors which typically need GPU acceleration. If slightly higher accuracy is required, OpenCV's DNN face detector (res10_300x300_ssd_iter, INT8-quantized) can be substituted, still executed through cv2.dnn on CPU with acceptable latency at the reduced input resolution.

Stage 4: Feature Extraction and Recognition

For the recognition step, the Local Binary Pattern Histogram (LBPH) algorithm — cv2.face.LBPHFaceRecognizer_create() — is used instead of a deep embedding network. LBPH encodes texture micro-patterns around each pixel into a histogram, which is inherently illumination-tolerant and computationally very light, making it well suited to constrained hardware. The recognizer is trained once (recognizer.train(faces, labels)) on the predefined dataset of enrolled faces, and at runtime recognizer.predict(face_roi) returns the closest matching identity with a confidence/distance score; a threshold rejects unknown faces.

Stage 5: Output

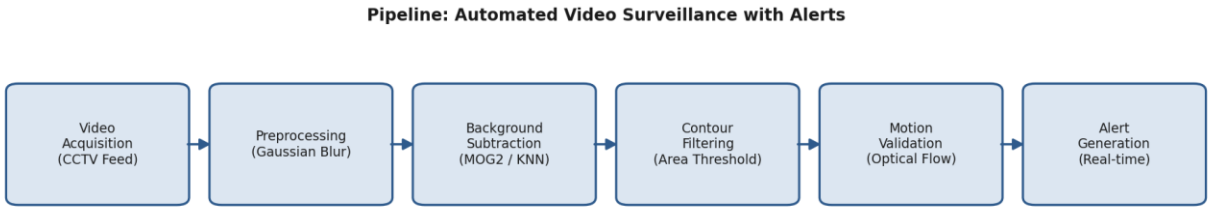
Detected faces are annotated using cv2.rectangle() for the bounding box and cv2.putText() for the predicted name and confidence score, then displayed or streamed onward for logging/access-control decisions.

Constraint Satisfaction Summary

Constraint	Design Decision
A. <30 FPS latency	Downscaled frames, threaded capture, Haar/LBP cascades and LBPH instead of deep CNNs keep per-frame processing time low
B. Varying lighting	CLAHE-based contrast normalization before detection and recognition
C. Limited memory/CPU	Classical, lightweight algorithms (Haar/LBP + LBPH) with small memory footprint; no GPU dependency
D. Predefined dataset recognition	LBPH model trained offline on the enrolled dataset; runtime does only fast histogram comparison

2. Automated Video Surveillance with Alert System

A security firm needs a system that detects motion/unusual activity, generates real-time alerts, works with standard CCTV feeds, and minimizes false positives in dynamic environments.



Stage 1: Video Acquisition

The standard CCTV stream is read with cv2.VideoCapture(rtsp_url) (RTSP/HTTP) or a local file/camera index. Frames are pulled in a dedicated thread/queue so that network jitter in the CCTV feed does not stall processing.

Stage 2: Preprocessing

Each frame is resized to a fixed working resolution and smoothed with cv2.GaussianBlur(frame, (21,21), 0) to suppress sensor noise and small pixel-level flicker that would otherwise be mistaken for motion.

Stage 3: Background Subtraction (Motion Detection)

cv2.createBackgroundSubtractorMOG2(history=500, varThreshold=16, detectShadows=True) (or the KNN variant) builds and continuously updates a statistical model of the static background. This adaptive model automatically absorbs slow, benign scene changes — swaying trees, gradual illumination drift, moving shadows — so only genuinely new foreground objects are flagged, directly addressing the dynamic-background constraint.

Stage 4: Contour Extraction and Noise Filtering

The foreground mask is passed through cv2.morphologyEx (opening/closing) to remove speckle noise, and cv2.findContours() extracts blob boundaries. Contours below a minimum cv2.contourArea() threshold are discarded, eliminating small, insignificant motion (e.g., insects, sensor noise) that generates false positives.

Stage 5: Motion Validation

For candidate regions that persist, dense or sparse optical flow — cv2.calcOpticalFlowFarneback() or cv2.calcOpticalFlowPyrLK() — is computed to confirm coherent directional motion rather than a one-frame flicker. Optionally a lightweight person/vehicle detector (HOG+SVM via cv2.HOGDescriptor, or a MobileNet-SSD through cv2.dnn) validates that the moving blob corresponds to a real object of interest before an alert is raised.

Stage 6: Alert Generation

An alert is only triggered when a validated motion region persists for N consecutive frames (temporal consistency check), which further suppresses transient false alarms. Alerts are

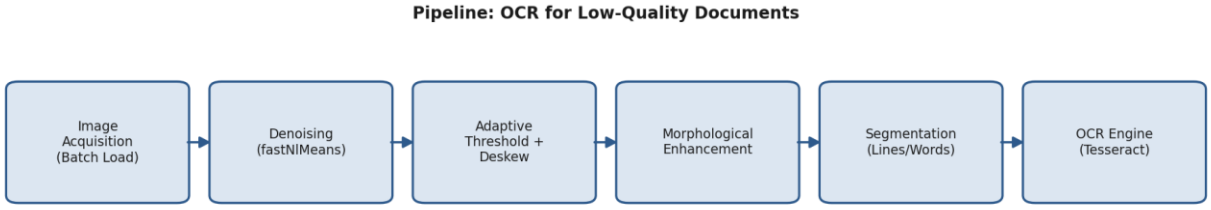
timestamped, logged, and can be pushed to a notification service; the frame is annotated with cv2.rectangle() around the activity region.

Constraint Satisfaction Summary

Constraint	Design Decision
A. Detect motion/unusual activity	MOG2 background subtraction + contour analysis
B. Real-time alerts	Lightweight background subtraction and contour filtering run comfortably above real-time frame rates
C. Standard CCTV feeds	cv2.VideoCapture supports RTSP/HTTP CCTV streams directly
D. Minimize false positives	Area thresholding, morphological noise removal, optical-flow validation and temporal persistence checks before alerting

3. OCR System for Low-Quality Documents

A digitization company needs to extract text from low-resolution, noisy scanned images, handle varying fonts and illumination, and process documents efficiently in batches.



Stage 1: Image Acquisition

Scanned documents are loaded in batch using cv2.imread() over a directory listing (glob), enabling the pipeline to process many files sequentially or in parallel via multiprocessing for throughput.

Stage 2: Denoising

Each image is converted to grayscale (cv2.cvtColor) and denoised using cv2.fastNlMeansDenoising(), which removes scanner/sensor noise while preserving character edges — critical for noisy low-resolution scans where simple blurring would smear thin strokes.

Stage 3: Adaptive Thresholding and Deskewing

Because illumination varies across the page (scanner shading, faded ink), cv2.adaptiveThreshold() with a Gaussian-weighted local threshold is used instead of a single global threshold, producing a clean binary image even under uneven lighting. Page skew is corrected by finding the dominant text angle via cv2.minAreaRect() on foreground pixels and rotating with cv2.warpAffine().

Stage 4: Morphological Enhancement

cv2.morphologyEx() with opening removes residual speckle noise, while a small closing/dilation operation reconnects character strokes broken by scan artifacts — important for handling varying/thin fonts before recognition.

Stage 5: Segmentation

cv2.findContours() combined with horizontal/vertical projection profiles segments the binarized page into lines, then words/characters, producing clean region-of-interest crops for the OCR engine and improving accuracy on dense or multi-column layouts.

Stage 6: Recognition and Batch Output

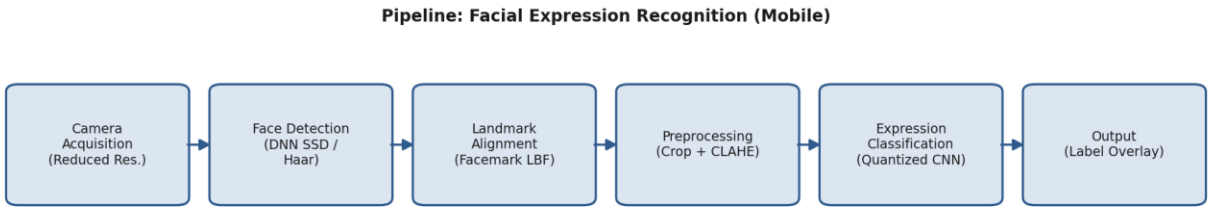
Each segmented region is passed to an OCR engine (e.g., pytesseract wrapping Tesseract) for character recognition; OpenCV performs all the image-quality-improving preprocessing that Tesseract itself does not do robustly. Results are aggregated per document and written out (text/CSV), with the batch loop parallelized using multiprocessing.Pool for efficient large-scale digitization.

Constraint Satisfaction Summary

Constraint	Design Decision
A. Low-resolution, noisy scans	Non-local-means denoising + morphological cleanup before recognition
B. Varying fonts and illumination	Adaptive (local) thresholding and CLAHE-based contrast handling instead of a single global threshold
C. Efficient batch processing	Vectorized OpenCV operations plus multiprocessing across the document batch

4. Facial Expression Recognition for Mobile Application

A mobile app must detect facial expressions (happy, sad, neutral) under limited mobile-CPU processing power, real-time performance requirements, and robustness to slight head movement and lighting changes.



Stage 1: Camera Acquisition

The mobile camera feed (via OpenCV's Android/iOS bindings) is captured at a reduced preview resolution (e.g., 320×240) to limit the data volume that later stages must process, directly addressing the limited-processing-power constraint.

Stage 2: Face Detection

A lightweight detector — either a Haar cascade or OpenCV's DNN SSD face detector (res10_300x300, quantized to INT8/FP16) run through cv2.dnn — locates the face bounding box each frame at low computational cost.

Stage 3: Landmark Alignment

Facial landmarks are located using OpenCV's Facemark LBF model (cv2.face.createFacemarkLBF()). The eyes/nose landmarks are used to compute an affine transform (cv2.warpAffine) that rotates and centers the face, so small head tilts or turns do not distort the features fed to the classifier — this is what gives robustness to slight head movement.

Stage 4: Preprocessing

The aligned face is cropped to the ROI, converted to grayscale, and normalized with CLAHE to counter lighting changes, then resized to a small fixed size (e.g., 48×48) — small enough to keep the classifier's compute cost low for a mobile CPU.

Stage 5: Expression Classification

A compact CNN (MobileNetV2/ShuffleNet-style architecture trained on an FER dataset) is exported and quantized (INT8) and executed on-device using cv2.dnn.readNetFromONNX()/readNetFromTensorflow(). Quantized lightweight CNNs give a strong accuracy-per-FLOP trade-off, which is the key to real-time performance on limited mobile hardware; a classical fallback (HOG features + linear SVM) can be used on very low-end devices for even lower latency at some accuracy cost.

Stage 6: Output

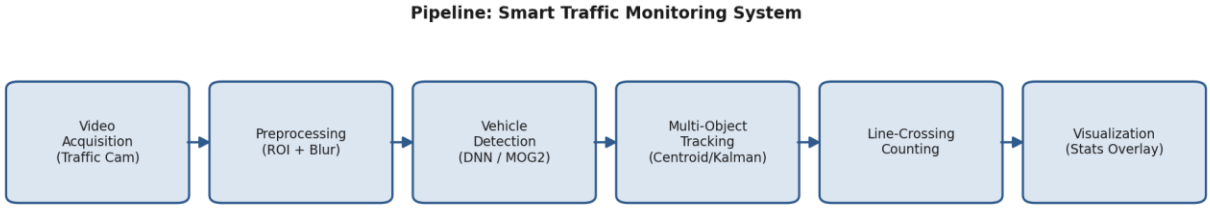
The predicted expression label and confidence are overlaid on the live camera preview using cv2.putText(), updated every processed frame.

Constraint Satisfaction Summary

Constraint	Design Decision
A. Limited mobile CPU	Small input resolution, quantized lightweight CNN (or HOG+SVM fallback) instead of a large deep network
B. Real-time performance	Low-cost detector + landmark alignment + compact classifier keep per-frame latency low
C. Robust to head movement/lighting	Landmark-based face alignment corrects pose; CLAHE normalizes illumination before classification

5. Smart Traffic Monitoring System

A city authority needs to detect vehicles from live video, track their movement across frames, count vehicles with statistics, and operate continuously with minimal delay.



Stage 1: Video Acquisition

Live traffic camera feed is captured via cv2.VideoCapture(), read in a background thread so frame grabbing never blocks the detection/tracking loop, supporting continuous 24/7 operation.

Stage 2: Preprocessing

Each frame is resized to a fixed processing resolution, mildly blurred with cv2.GaussianBlur() to suppress noise, and a Region of Interest mask restricts analysis to the road area, reducing unnecessary computation on irrelevant scene regions (sidewalks, sky) and speeding up the pipeline.

Stage 3: Vehicle Detection

For robust detection across vehicle types, occlusion and varying scale, a pretrained deep detector (YOLO or MobileNet-SSD) is run through cv2.dnn.readNet(), which gives GPU/CPU-accelerated inference directly within OpenCV. Where hardware is very constrained, cv2.createBackgroundSubtractorMOG2() with contour filtering offers a lighter-weight alternative for a fixed-camera scene, trading some accuracy for speed.

Stage 4: Multi-Object Tracking

Detected vehicle boxes are handed to a tracker that assigns and maintains a persistent ID per vehicle across frames — either a centroid-tracking algorithm (Euclidean distance matching between frames) or OpenCV's built-in trackers (cv2.TrackerCSRT_create()/cv2.TrackerKCF_create()), optionally smoothed with a Kalman filter to predict position during brief occlusion and keep trajectories continuous.

Stage 5: Line-Crossing Counting

A virtual counting line is defined across the road (cv2.line() for visualization). When a tracked vehicle's centroid crosses this line (checked frame-to-frame using its unique ID), the count for that lane/direction is incremented exactly once per vehicle — the persistent ID from the tracking stage is what prevents double counting.

Stage 6: Visualization and Statistics

cv2.rectangle() draws each vehicle's bounding box, cv2.putText() overlays its ID and (optionally) an estimated speed from pixel displacement over time given camera calibration, and a running dashboard overlay reports cumulative counts per direction/vehicle class in real time.

Constraint Satisfaction Summary

Constraint	Design Decision
A. Detect vehicles from live streams	DNN-based detector (YOLO/MobileNet-SSD) or MOG2 background subtraction via cv2.dnn
B. Track movement across frames	Centroid tracker / CSRT-KCF tracker with Kalman-filter smoothing maintains persistent per-vehicle IDs
C. Count vehicles and display statistics	Line-crossing logic keyed on tracked IDs avoids double counting; overlay shows live statistics
D. Continuous operation, minimal delay	Threaded video capture, ROI masking and an efficient detector/tracker pairing keep per-frame latency low for 24/7 operation